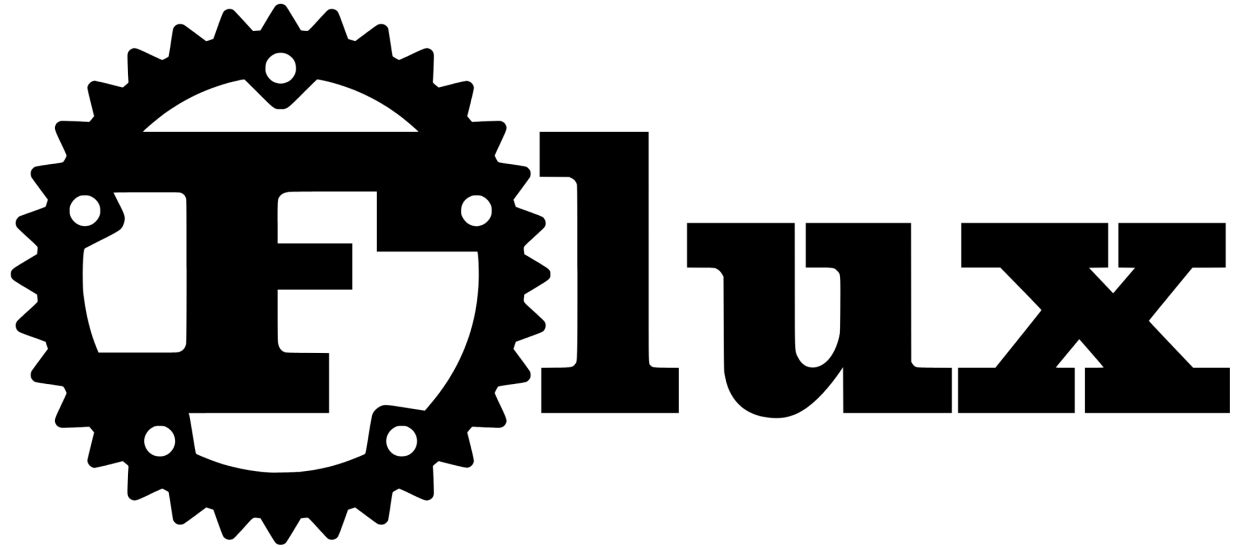


Program Verification for the 21st Century



Ranjit Jhala, *UC San Diego*

Hoare Logic (1969)

An Axiomatic Basis for Computer Programming

C. A. R. HOARE

The Queen's University of Belfast, Northern Ireland*

In this paper an attempt is made to explore the logical foundations of computer programming by use of techniques which were first applied in the study of geometry and have later been extended to other branches of mathematics. This involves the elucidation of sets of axioms and rules of inference which can be used in proofs of the properties of computer programs. Examples are given of such axioms and rules, and a formal proof of a simple theorem is displayed. Finally, it is argued that important advantages, both theoretical and practical, may follow from a pursuance of these topics.

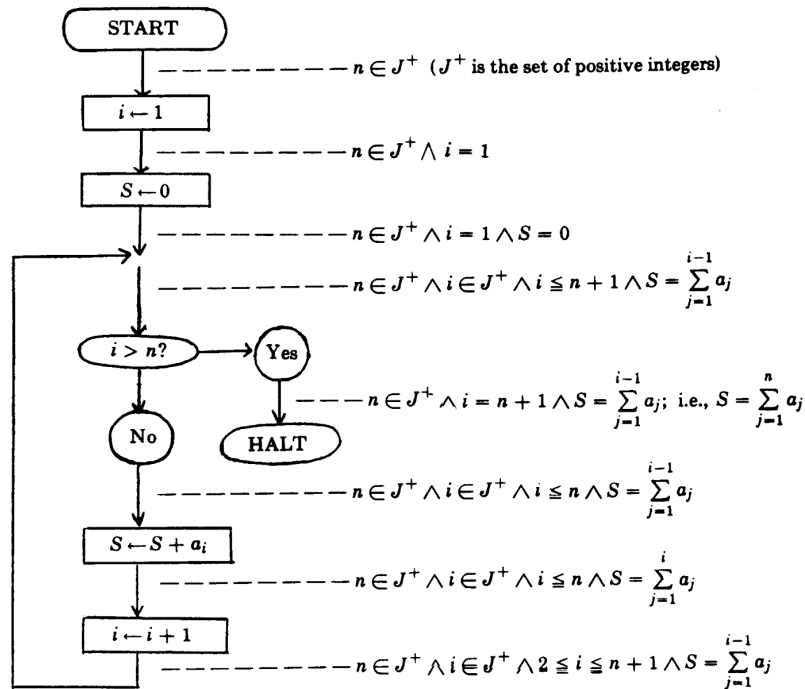
Hoare Logic (1969)

$$\{P\} \quad c \quad \{Q\}$$

Precondition Command Postcondition

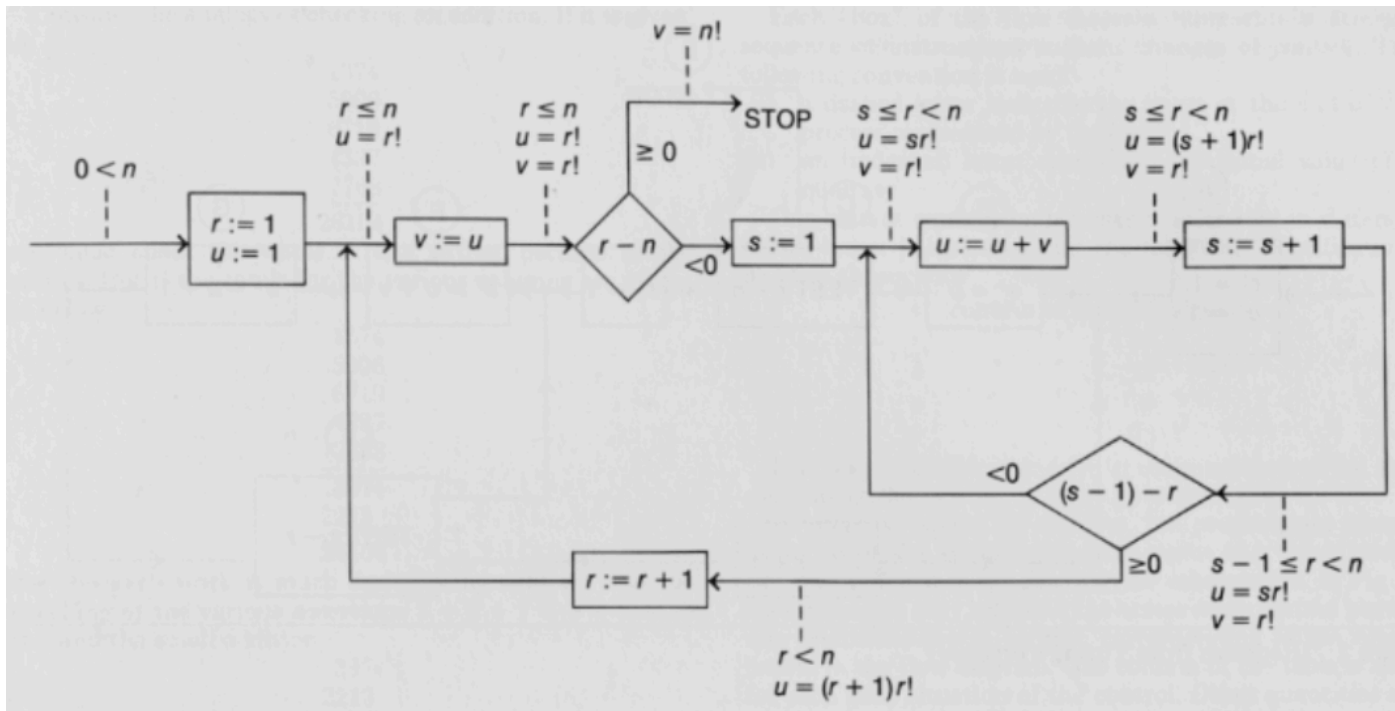
Floyd (1967)

“Assigning Meanings to Programs”



Turing (1949)

“Checking a Large Routine”



The Problem

Program Logic vs. 21st Century Code

Program Logic[★] *vs.* 21st Century Code

Program Logic[★] *vs.* 21st Century Code

Iterations,

Program Logic[★] *vs.* 21st Century Code

Iterations, Collections,

Program Logic[★] *vs.* 21st Century Code

Iterations, Collections, λ -Abstractions, ...

Program Logic[★] *vs.* 21st Century Code

Iterations, Collections, λ -Abstractions, ...

[★] Illustrated using Verus, a Rust verifier similar to Dafny

Program Logic *vs.* 21st Century Code

Iterations, Collections, λ -Abstractions, ...

Iteration using while

```
fn dot_product(x: &Vec<f64>, y: &Vec<f64>) → f64
{
    let mut sum = 0.0;
    let mut i = 0;
    while i < x.len()
    {
        sum += x[i] * y[i];
        i += 1;
    }
    sum
}
```

Goal: Verify vector access *in-bounds*

Iteration using while

```
fn dot_product(x: &Vec<f64>, y: &Vec<f64>) → f64
  requires x.len() == y.len(),
{
  let mut sum = 0.0;
  let mut i = 0;
  while i < x.len()
    invariant i <= x.len() && x.len() == y.len()
  {
    sum += x[i] * y[i];
    i += 1;
  }
  sum
}
```

Feasible: Need invariant on *explicit* iterator state!

Iteration using for

```
fn dot_product(x: &Vec<f64>, y: &Vec<f64>) → f64
  requires x.len() == y.len(),
{
  let mut sum = 0.0;
  for i in 0..x.len()
    invariant ???
  {
    sum += x[i] * y[i];
  }
  sum
}
```

Problem: Need invariant on *implicit* iterator state!

Program Logic *vs.* 21st Century Code

Iterations, *Collections*, λ -Abstractions, ...

Collections

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>) → Vec<f64>

{
    let mut res = Vec::new();
    let mut i = 0;
    while i < ws.len()

    {
        let weighted_input = dot_product(&ws[i], ins);
        res.push(sigmoid(weighted_input));
        i += 1;
    }
    res
}
```

Goal: Verify *precondition* of dot_product

Collections

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>) → Vec<f64>
  requires forall |i: int| 0 <= i < ws.len() ==> ws[i].len() == ins.len()
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < ws.len()
    invariant
      i ≤ ws.len(),
      forall |j: int| 0 <= j < ws.len() ==> ws[j].len() == ins.len(),
    {
      let weighted_input = dot_product(&ws[i], ins);
      res.push(sigmoid(weighted_input));
      i += 1;
    }
  res
}
```

Problem: Quantified contracts (hard to write & verify)

Program Logic *vs.* 21st Century Code

Iterations, Collections, *λ -Abstractions*, ...

λ -Abstractions

```
// Create a 2-d out_size x in_size weight matrix
```

λ -Abstractions

```
// Create a 2-d out_size x in_size weight matrix
fn mk_weights(in_size: usize, out_size: usize) → Vec<Vec<f64>> {
    init(out_size, |_| {
        init(in_size, |_| random())
    })
}
```

λ -Abstractions

```
// Create a 2-d out_size x in_size weight matrix
fn mk_weights(in_size: usize, out_size: usize) → Vec<Vec<f64>> {
    init(out_size, |_| {
        init(in_size, |_| random())
    })
}

fn init<A>(n: usize, f: impl FnMut(usize) → A) → Vec<A> {
    let mut res = Vec::new();
    for i in 0 .. n {
        res.push(f(i));
    }
    res
}
```

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>

{
    let mut res = Vec::new();
    let mut i = 0;
    while i < n

        {
            res.push(f(i));
            i += 1;
        }
    res
}
```

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>
  requires
    forall|i: int| 0 <= i < n ==> call_requires(f, i),
  ensures
    res.len() = n,
    forall|i: int| 0 <= i < n ==> call_ensures(f, i, res[i])
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < n
    invariant
      i ≤ n,
      res.len() = i,
      forall|i: int| 0 <= i < n ==> call_requires(f, i),
      forall|j: int| 0 <= j < i ==> call_ensures(f, j, res[j])
    {
      res.push(f(i));
      i += 1;
    }
  res
}
```

Problem: Hairy *higher-order* contracts!

Program Logic *vs.* 21st Century Code

Iterations, Collections, λ -Abstractions, ...

1. Problem

Program Logic *vs.* 21st century features

1. Problem

Program Logic vs. 21st century features

2: Solution

Refinements: *Weave* assertions into types

Refinements

Weave **Assertions** into **Types**

Weave *Assertions* into *Types*

1. *Refinements* `i32`, `bool`, ...

2. *Ownership* `mut`, `&`, `&mut`, ...

3. *Example* revisited

1. *Refinements*

Index specifies *single value*

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

Index specifies *single value*

Index specifies *single value*

B [*v*]

Base Type *Refine* Index

Index specifies *single value*

i32[**5**]

The *singleton* **i32** that is equal to **5**

Index specifies *single value*

`bool[true]`

The *singleton* `bool` that is equal to `true`

Index specifies *single value*

```
fn tt() → bool[true] {  
    1 < 2  
}
```

Output type specifies *Postcondition*

A function that *always returns* **true**

Index specifies *single value*

```
fn ff() → bool[false] {  
    2 < 1  
}
```

Output type specifies *Postcondition*

A function that *always returns* false

Index specifies *single value*

```
fn twelve() → i32[12] {  
    4 + 8  
}
```

Output type specifies *Postcondition*

A function that *always returns* 12

Index specifies *single value*

```
fn assert(b:bool[true]){} }
```

Input type specifies *Precondition*

A function that *requires* input be **true**

Index specifies *single value*

```
fn assert(b:bool[true]){}  
  
...  
assert(1 < 2);  
assert(10 < 2); // flux error!
```

Input type specifies *Precondition*

A function that *requires* input be **true**

Index specifies *single value*

Constants are *boring!*

Index specifies *single value*

Constants are *boring*!

Parameters abstract over *input values*

Parameters abstract over *input values*

Refinement parameters

```
forall<n: int> fn (i32[n]) → bool[n > 0]
```

Parameters abstract over *input values*

Refinement parameters

`forall<n: int> fn (i32[n]) → bool[n > 0]`

Declare with @-syntax

`fn (i32[@n]) → bool[n > 0]`

Parameters abstract over *input values*

Refinement parameters

```
forall<n: int> fn (i32[n]) → bool[n > 0]
```

Declare with @-syntax

```
fn (i32[@n]) → bool[n > 0]
```

Or desugar from Rust

```
fn (n:i32) → bool[n > 0]
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
  n > 0  
}
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
    n > 0  
}  
  
...  
assert(is_pos(5));           // ok
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
    n > 0  
}  
  
...  
assert(is_pos(5));           // ok  
assert(is_pos(5 - 8));      // error
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
  n + 1  
}
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
    n + 1  
}  
  
...  
assert(incr(5 - 5) > 0); // ok
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
    n + 1  
}  
  
...  
assert(incr(5 - 5) > 0); // ok  
assert(incr(5 - 6) > 0); // error
```

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

Index specifies *single value*

B[*v*]

Index specifies *single value*

B [*v*]

But what if we *don't know* the exact value?

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

Existential specifies *sets of values*

Constable & Smith 1987, Rushby et al. 1997

Existential specifies *sets of values*

$$\mathbf{B} \{ v : p(v) \}$$

Base Type Constraint

Existential specifies *sets of values*

$$\mathbf{B} \{ v : p(v) \}$$

Set of $\mathbf{B}$ values whose index v satisfies $p(v)$

Existential specifies *sets of values*

i32 { $v : 0 \leq v$ }

i32 values that are *non-negative*

Existential specifies *sets of values*

`usize` $\{v : v < n\}$

`usize` values *less than* `n`

Existential specifies *sets of values*

`abs` returns a non-negative `i32`

```
fn abs(n:i32) → i32{v:0 ≤ v} {  
  if n < 0 {  
    0 - n  
  } else {  
    n  
  }  
}
```

Existential specifies *sets of values*

abs returns a non-negative **i32**

```
fn abs(n:i32) → i32{v:0 ≤ v} {  
  if n < 0 {  
    0 - n  
  } else {  
    n  
  }  
}  
...  
assert(abs(n) ≥ 0);
```

Exercise: How can we *fix* the spec for *abs*?

Existential specifies *sets of values*

abs returns a non-negative **i32**

```
fn abs(n:i32) → i32{v:0 ≤ v} {  
  if n < 0 {  
    0 - n  
  } else {  
    n  
  }  
}  
...  
assert(abs(n) ≥ 0);  
assert(abs(n) >= n); // EX: How to fix?
```

Exercise: How can we *fix* the spec for abs?

Existential specifies *sets of values*

get requires a valid index!

```
fn get<T>(x: &[T], i: usize) → &T
{
    &x[i] // EX: How to fix?
}
```

Exercise: What is a suitable type for the input *i*?

Existential specifies *sets of values*

get requires a valid index!

```
fn get<T>(x: &[T][@n], i: usize{v:v<n}) → &T
{
    &x[i]
}
```

Precondition: *i less than size of slice n*

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

Weave **Assertions** into **Types**

1. *Refinements* `i32`, `bool`, ...

2. *Ownership* `mut`, `&`, `&mut`, ...

3. *Example* revisited

1. Problem

Program Logic vs. 21st century features

2: Solution

Refinements: *Weave* assertions into types

2. Ownership

2. *Ownership*

Update types at *owned locations*

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrow*s

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

Update types at *owned locations*

Update types at *owned locations*

```
let mut x = 0;    // x : i32[0]  
assert(x == 0);
```

Exclusive ownership allows strong updates

Update types at *owned locations*

```
let mut x = 0;      // x : i32[0]
assert(x == 0);
x += 10;            // x : i32[10]
assert(x == 10);
```

Exclusive ownership allows strong updates

Update types at *owned locations*

```
let mut x = 0;      // x : i32[0]
assert(x == 0);
x += 10;            // x : i32[10]
assert(x == 10);
x += 10;            // x : i32[20]
assert(x == 20);
```

Exclusive ownership allows strong updates

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

Preserve refinements in *borrows*

Preserve refinements in *borrow*s

```
fn read(x: &i32{v: 0 < v}) {  
    let n = *x;  
    assert(0 < n);  
}
```

& reference = aliasing but *no mutation*

Value *read* through & reference satisfies refinements

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

Strong updates at *mutable borrows*

Strong updates at *mutable borrows*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}
```

Strong updates at *mutable borrows*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}  
fn test() {  
    let mut z = 1; // z: i32[1]
```

Strong updates at *mutable borrows*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}  
  
fn test() {  
    let mut z = 1; // z: i32[1]  
    incr(&z);  
    assert(z == 2); // z: i32[2]  
}
```

Strong updates at *mutable borrows*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}  
  
fn test() {  
    let mut z = 1; // z: i32[1]  
    incr(&z);  
    assert(z == 2); // z: i32[2]  
}
```

Sound as &mut is *unique* (no aliasing)

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

Weave *Assertions* into *Types*

1. *Refinements* `i32`, `bool`, ...

2. *Ownership* `mut`, `&`, `&mut`, ...

3. *Example* revisited

3. Example revisited

Types make specification & verification *compositional*

struct: *Refined Vectors*

```
struct RVec<T> {  
    inner: Vec<T>;  
}
```

RVec is a *refined wrapper* around standard library Vec

struct: *Refined Vectors*

```
#[refined_by(len: int)]  
struct RVec<T> {  
    inner: Vec<T>;  
}
```

refined_by: *refinement value(s)* tracked for RVec<T>

Refined Vectors: Specification

Refined Vectors: *Specification*

```
impl RVec<T> {
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
}
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
  
    // retrieve the size of the vector  
    fn len(&self[@n]) → usize[n];  
}
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
  
    // retrieve the size of the vector  
    fn len(&self[@n]) → usize[n];  
  
    // get/set element at index i statically checking bounds  
    fn get(&self[@n], i: usize{v: v < n}) → &T;  
    fn set(&mut self[@n], i: usize{v: v < n}, val: T) → &T;
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
  
    // retrieve the size of the vector  
    fn len(&self[@n]) → usize[n];  
  
    // get/set element at index i statically checking bounds  
    fn get(&self[@n], i: usize{v: v < n}) → &T;  
    fn set(&mut self[@n], i: usize{v: v < n}, val: T) → &T;  
  
    // insert element at the end of the vector  
    fn push(&mut self[@n], val: T) ensures self: RVec<T>[n+1];  
}
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
  
    // retrieve the size of the vector  
    fn len(&self[@n]) → usize[n];  
  
    // get/set element at index i statically checking bounds  
    fn get(&self[@n], i: usize{v: v < n}) → &T;  
    fn set(&mut self[@n], i: usize{v: v < n}, val: T) → &T;  
  
    // insert element at the end of the vector  
    fn push(&mut self[@n], val: T) ensures self: RVec<T>[n+1];  
}
```

21st Century Verification

Iterations, Collections, λ -Abstractions, ...

Iteration using while

```
fn dot_product(x:&Vec<f64>, y:&Vec<f64>)
    → f64
    requires x.len() == y.len(),
{
}
}
```

```
fn dot_product(x:&Vec<f64>[@n], y:&Vec<f64>[n])
    → f64
{
}
}
```

Same index specifies same size precondition

Iteration using while

```
fn dot_product(x:&Vec<f64>, y:&Vec<f64>)
  → f64
  requires x.len() == y.len(),
{
  let mut sum = 0.0;
  let mut i = 0;
  while i < x.len()
    invariant
      i <= x.len(),
      x.len() == y.len()
  {
    sum += x[i] * y[i];
    i += 1;
  }
  sum
}
```

```
fn dot_product(x:&Vec<f64>[@n], y:&Vec<f64>[n])
  → f64
{
  let mut sum = 0.0;
  let mut i = 0;
  while i < x.len()
  {
    sum += x[i] * y[i];
    i += 1;
  }
  sum
}
```

Invariant via *immutable type*

Iteration with **for** loop

```
fn dot_product(x:&Vec<f64>[@n], y:&Vec<f64>[n]) -> f64
{
    let mut sum = 0.0;
    for i in 0..x.len() {    // iterator guarantees
        sum += x[i] * y[i];  // 0 ≤ i < n
    }
    sum
}
```

Permits *range* iteration instead of old school while

21st Century Verification

Iterations, *Collections*, λ -Abstractions, Datatypes,...

Collections

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>)
  → Vec<f64>
  requires
     $\forall i: \text{int} \mid 0 \leq i < \text{ws.len()} \Rightarrow \text{ws}[i].\text{len()} == \text{ins.len()}()$ 
{
}
```

[illegible]

Quantified contracts via *type composition*

Collections

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>)
  → Vec<f64>
  requires
     $\forall |i: \text{int}| \ 0 \leq i < \text{ws.len()} \Rightarrow \text{ws}[i].\text{len()} = \text{ins.len()}$ 
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < ws.len()
    invariant
       $i \leq \text{ws.len()},$ 
       $\forall |j: \text{int}| \ 0 \leq j < \text{ws.len()} \Rightarrow \text{ws}[j].\text{len()} == \text{ins.len()}$ 
    {
      let weighted_input = dot_product(&ws[i], ins);
      res.push(sigmoid(weighted_input));
      i += 1;
    }
  res
}
```

```
fn forward(ws: &Vec<Vec<f64>[n]>, ins: &Vec<f64>[@n])
  → Vec<f64>
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < ws.len()
  {
    let weighted_input = dot_product(&ws[i], ins);
    res.push(sigmoid(weighted_input));
    i += 1;
  }
  res
}
```

Quantified invariants via *syntactic typing*

21st Century Verification

Iterations, Collections, *λ -Abstractions*, Datatypes, ...

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>
  requires
    ∀ |i:int| 0 ≤ i < n ==> call_requires(f, i)
  ensures
    res.len() == n,
    ∀ |i:int| 0 ≤ i < n ==> call_ensures(f, i, res[i])
{
}
```

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]
{
}
```

Higher-order contracts via *parametric polymorphism*

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>
  requires
     $\forall |i: \text{int}| 0 \leq i < n \Rightarrow \text{call\_requires}(f, i)$ 
  ensures
    res.len() == n,
     $\forall |i: \text{int}| 0 \leq i < n \Rightarrow \text{call\_ensures}(f, i, \text{res}[i])$ 
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < n
    invariant
       $i \leq n, \text{res.len()} == i,$ 
       $\forall |i: \text{int}| 0 \leq i < n \Rightarrow \text{call\_requires}(f, i),$ 
       $\forall |j: \text{int}| 0 \leq j < i \Rightarrow \text{call\_ensures}(f, j, \text{res}[j])$ 
    {
      res.push(f(i));
      i += 1;
    }
  res
}
```

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]
{
  let mut res = Vec::new(); // res: Vec<A>[0]
  let mut i = 0;
  while i < n                // res: Vec<A>[i]

  {
    res.push(f(i));
    i += 1;
  }
  res                        // res: Vec<A>[n]
}
```

Quantifier-free Invariant via *refinement synthesis*

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]  
{ /* ... */ }
```

Polymorphic Instantiation via *refinement synthesis*

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]
{ /* ... */ }

// Create a 2-d out_size x in_size weight matrix
fn mk_weights(in_size: usize, out_size: usize)
  → Vec<Vec<f64>[in_size]>[out_size]
{
  init(out_size, |_| { // A := Vec<f64>[in_size]
    init(in_size, |_| random()) // A := f64
  })
}
```

Polymorphic Instantiation via *refinement synthesis*

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]
{ /* ... */ }

// Create a 2-d out_size x in_size weight matrix
fn mk_weights(in_size: usize, out_size: usize)
  → Vec<Vec<f64>[in_size]>[out_size]
{
  init(out_size, |_| { // A := Vec<f64>[in_size]
    init(in_size, |_| random()) // A := f64
  })
}
```

Polymorphic Instantiation via *refinement synthesis*

21st Century Verification

Iterations, Collections, λ -Abstractions, *Datatypes*, ...

Datatypes

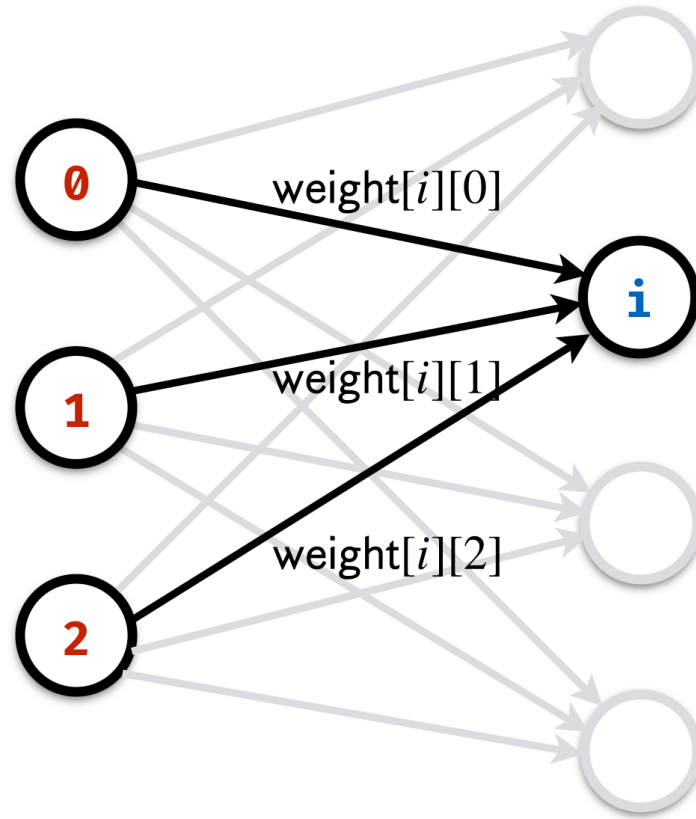
`struct`

“Product” types

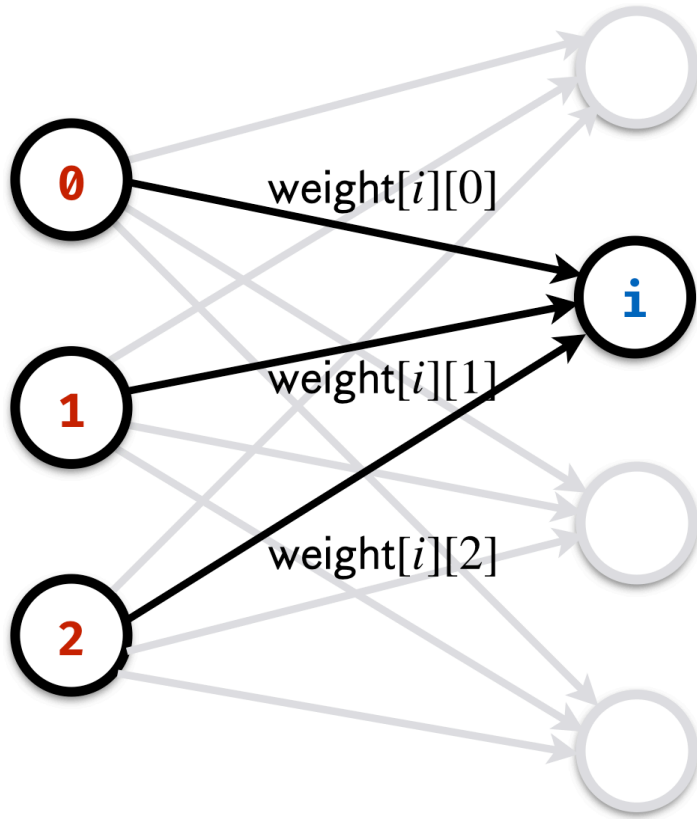
`enum`

“Sum” types

Example: *Neuron Layer*

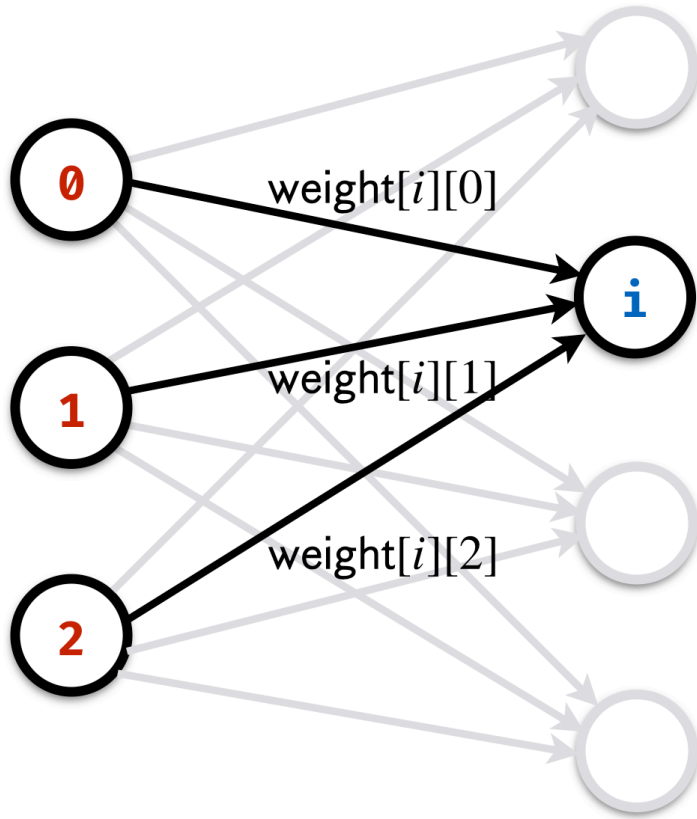


Neuron Layer: *Specification*



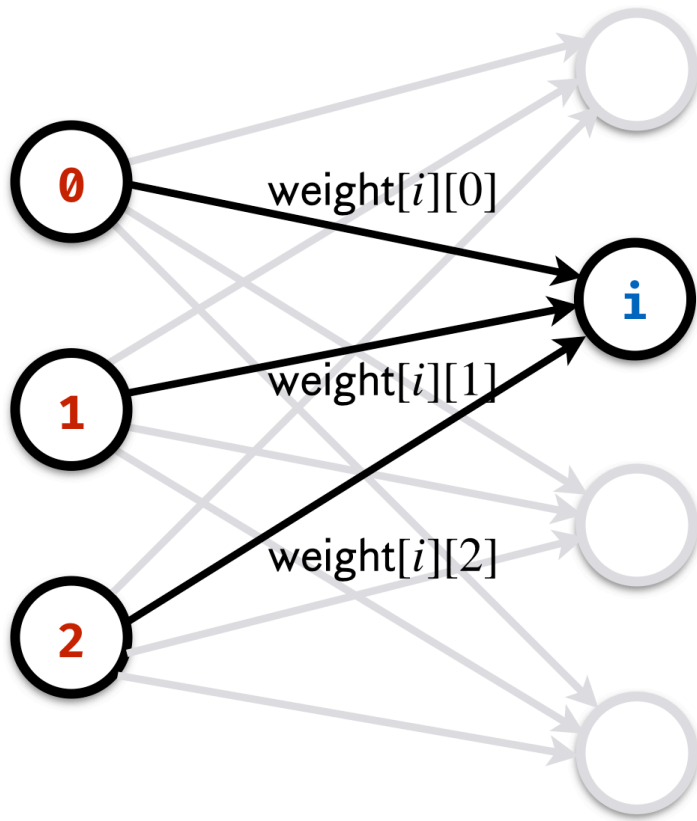
```
#[refined_by(i: int, o: int)]  
struct Layer {
```

Neuron Layer: *Specification*



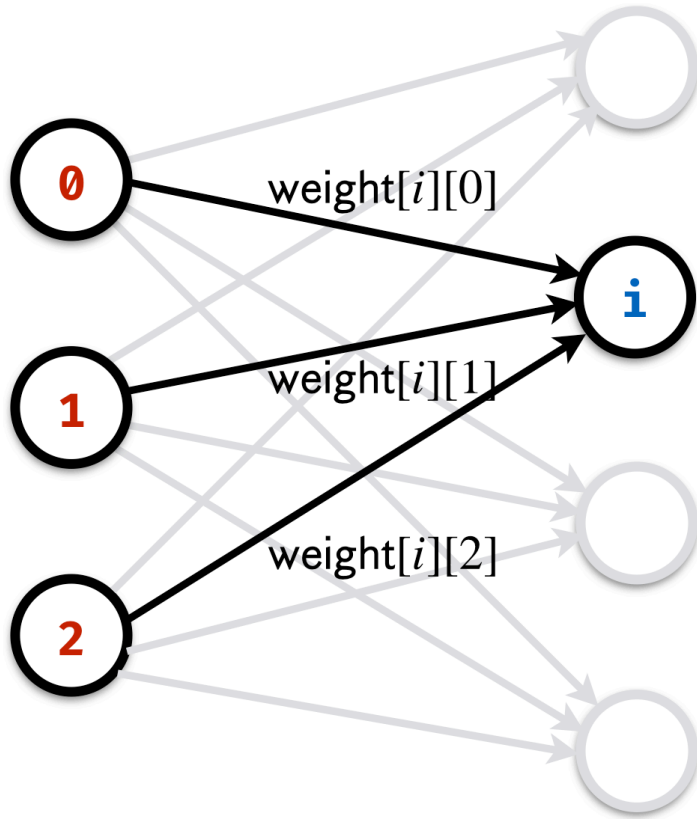
```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],
```

Neuron Layer: *Specification*



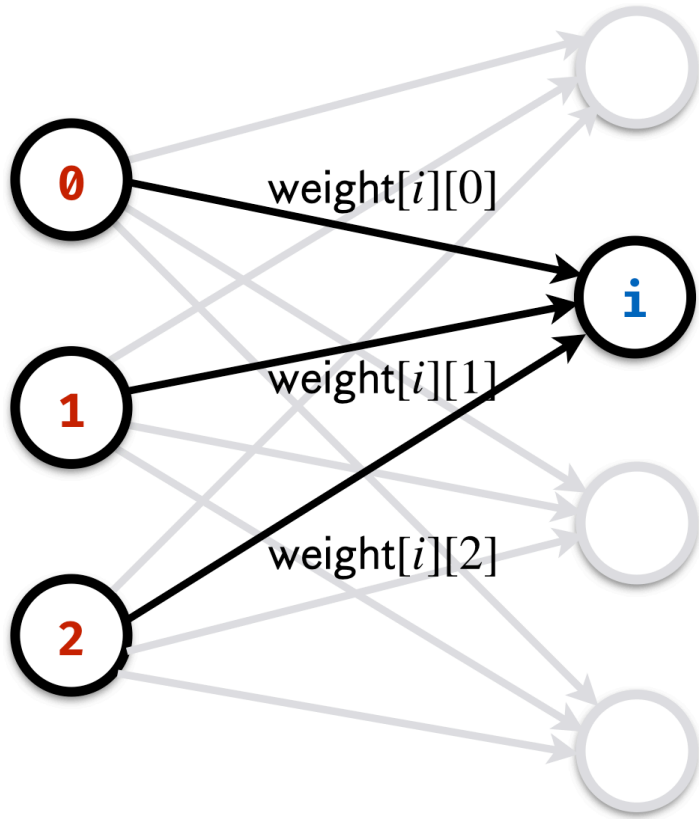
```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],
```

Neuron Layer: *Specification*



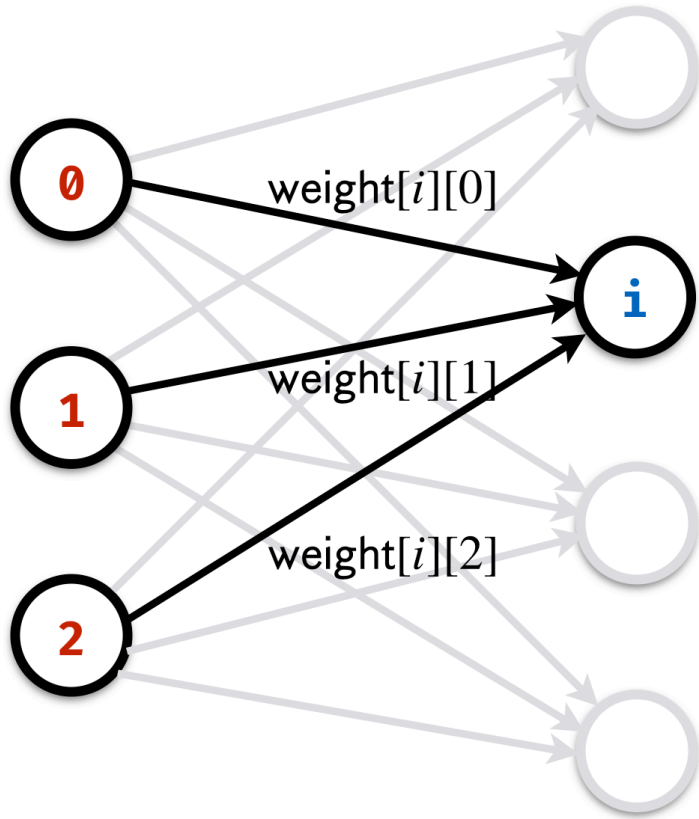
```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],  
    bias: RVec<f64>[o],  
}
```

Neuron Layer: *Specification*



```
#[refined_by(i: int, o: int)]
struct Layer {
    num_inputs: usize[i],
    num_outputs: usize[o],
    weight: RVec<RVec<f64>[i]>[o],
    bias: RVec<f64>[o],
    outputs: RVec<f64>[o],
}
```

Neuron Layer: *Specification*

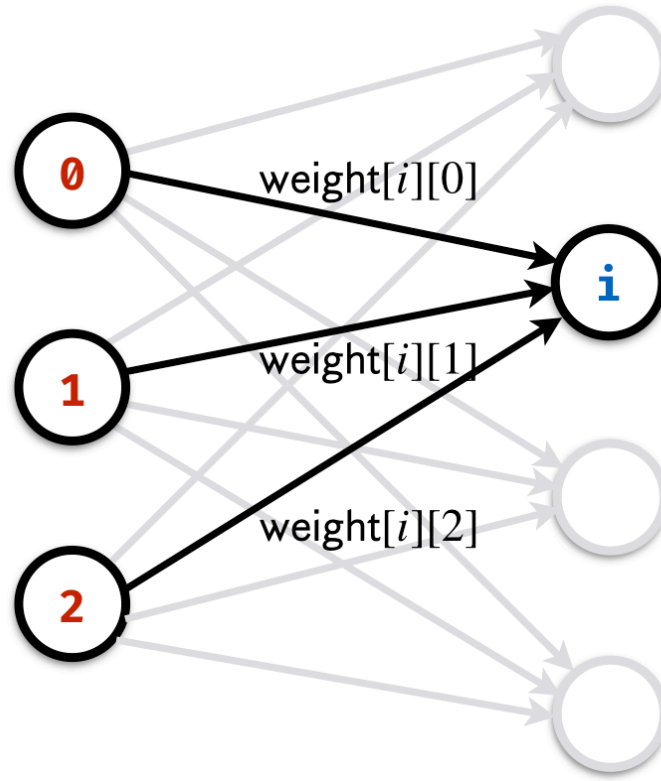


```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],  
    bias: RVec<f64>[o],  
    outputs: RVec<f64>[o],  
}
```

Neuron Layer: *Verification*

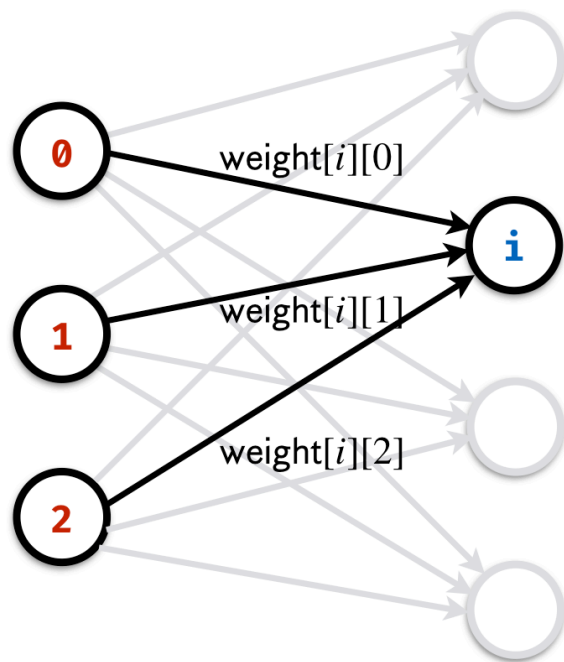
```
fn new(i: usize, o: usize) → Layer[i, o] {  
  let mut rng = rand::thread_rng();  
  Layer {  
    num_inputs: i,  
    num_outputs: o,  
    weight: init(o, |_| init(i, |_| rng.gen_range(-1.0..1.0))),  
    bias: init(o, |_| rng.gen_range(-1.0..1.0)),  
    outputs: init(o, |_| 0.0),  
  }  
}
```

Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

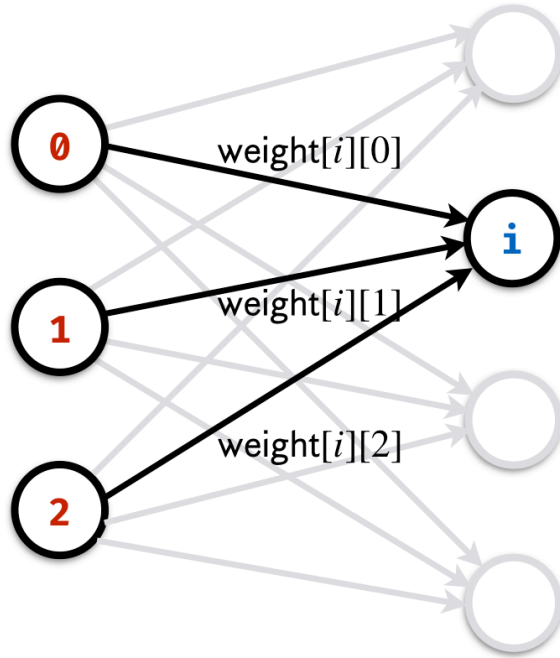
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

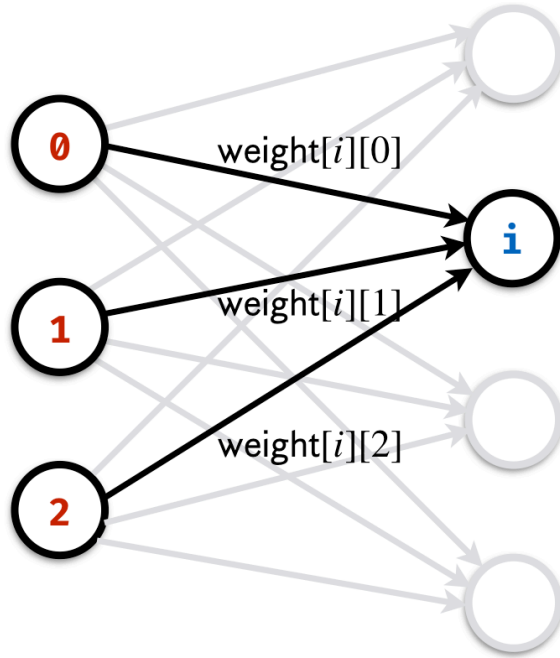
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

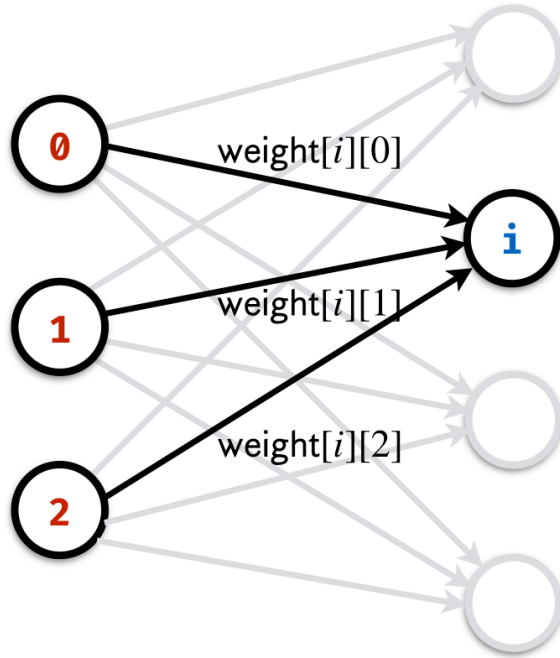
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

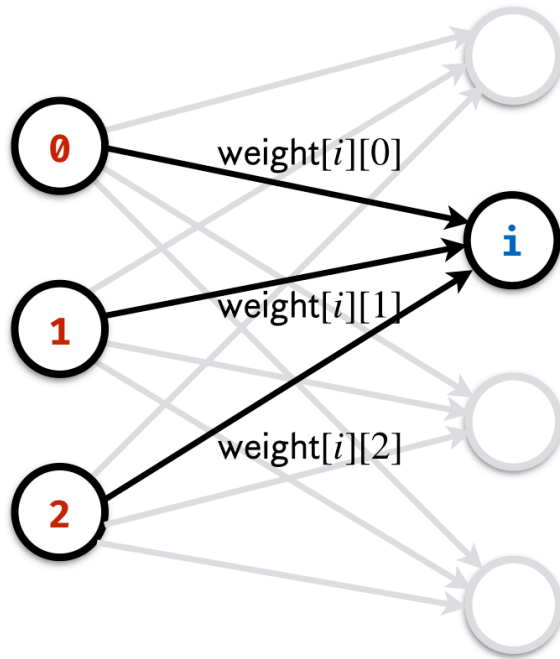
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
    })  
}
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

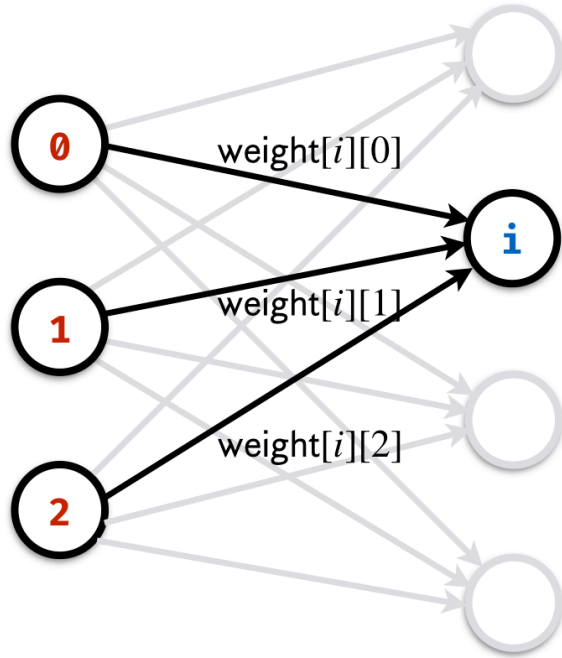
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

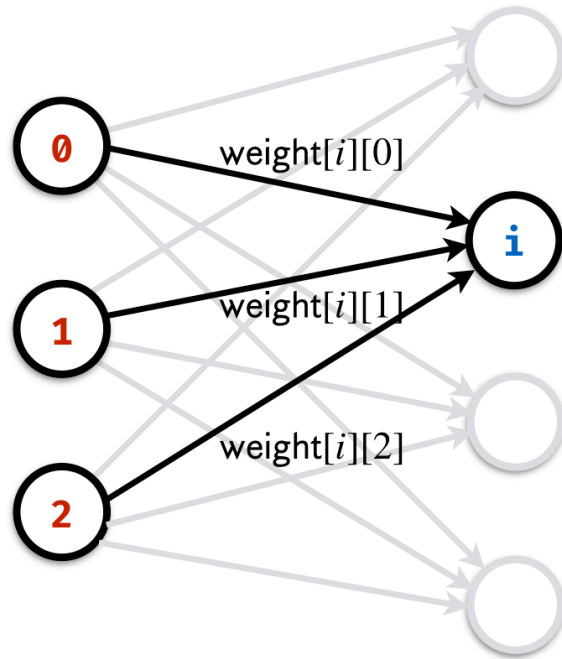
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

Exercise: Can you *fix* the error?

Datatypes

struct

“Product” types

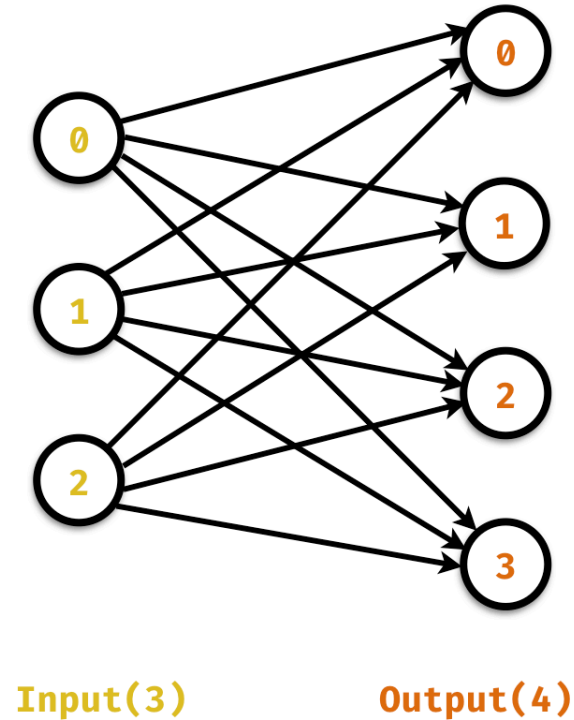
enum

“Sum” types

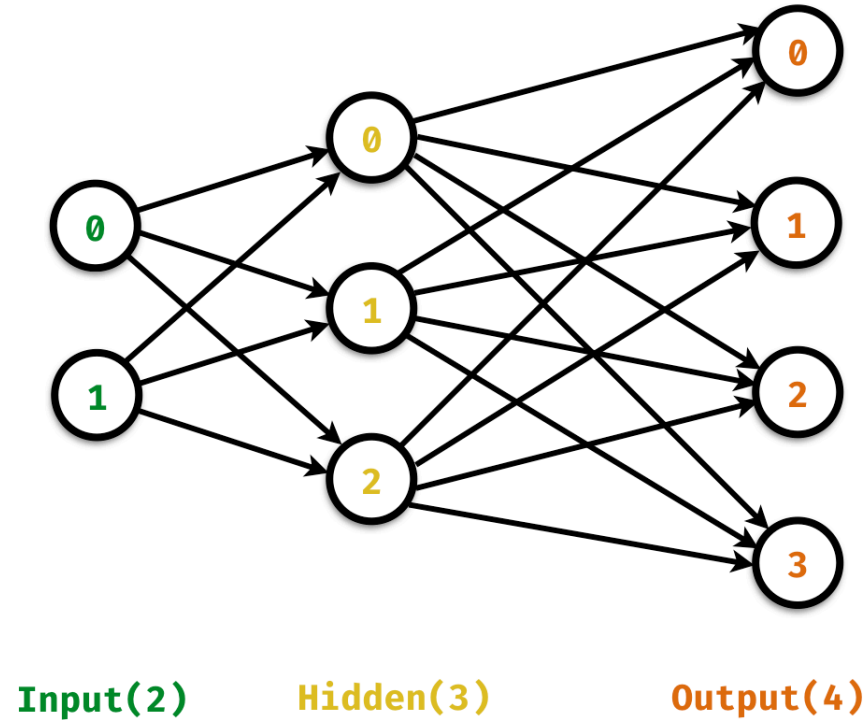
enum

Example: *Neural Network*

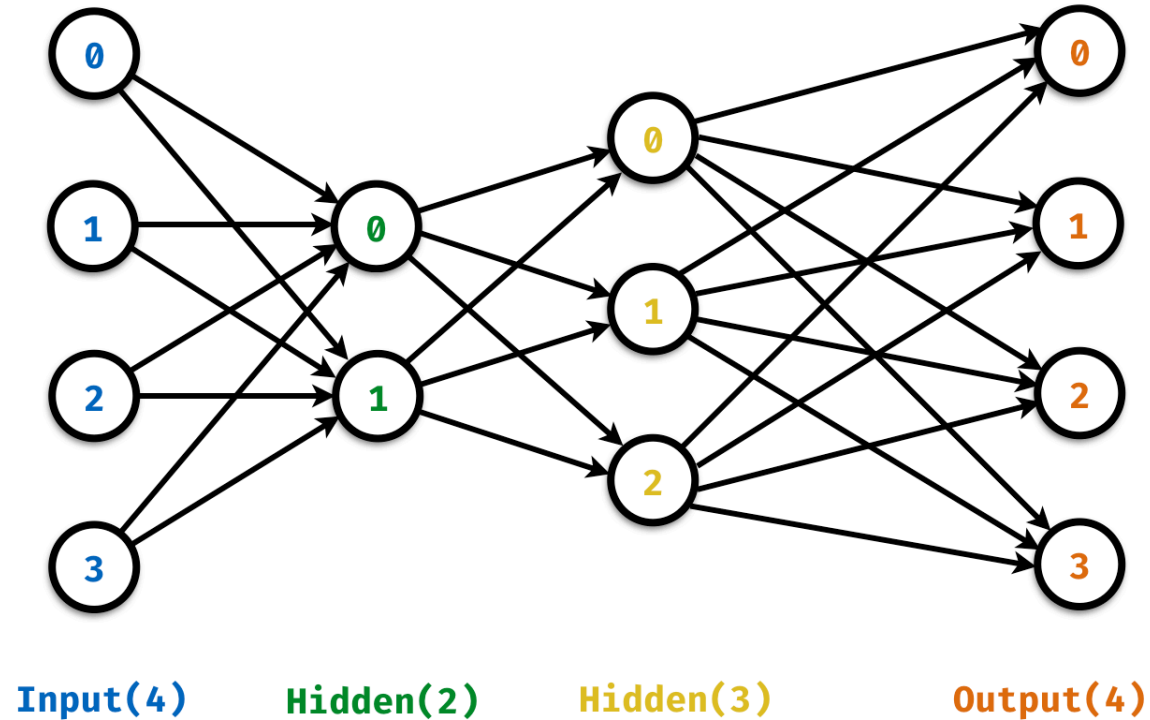
Neural Network has *Many* Layers



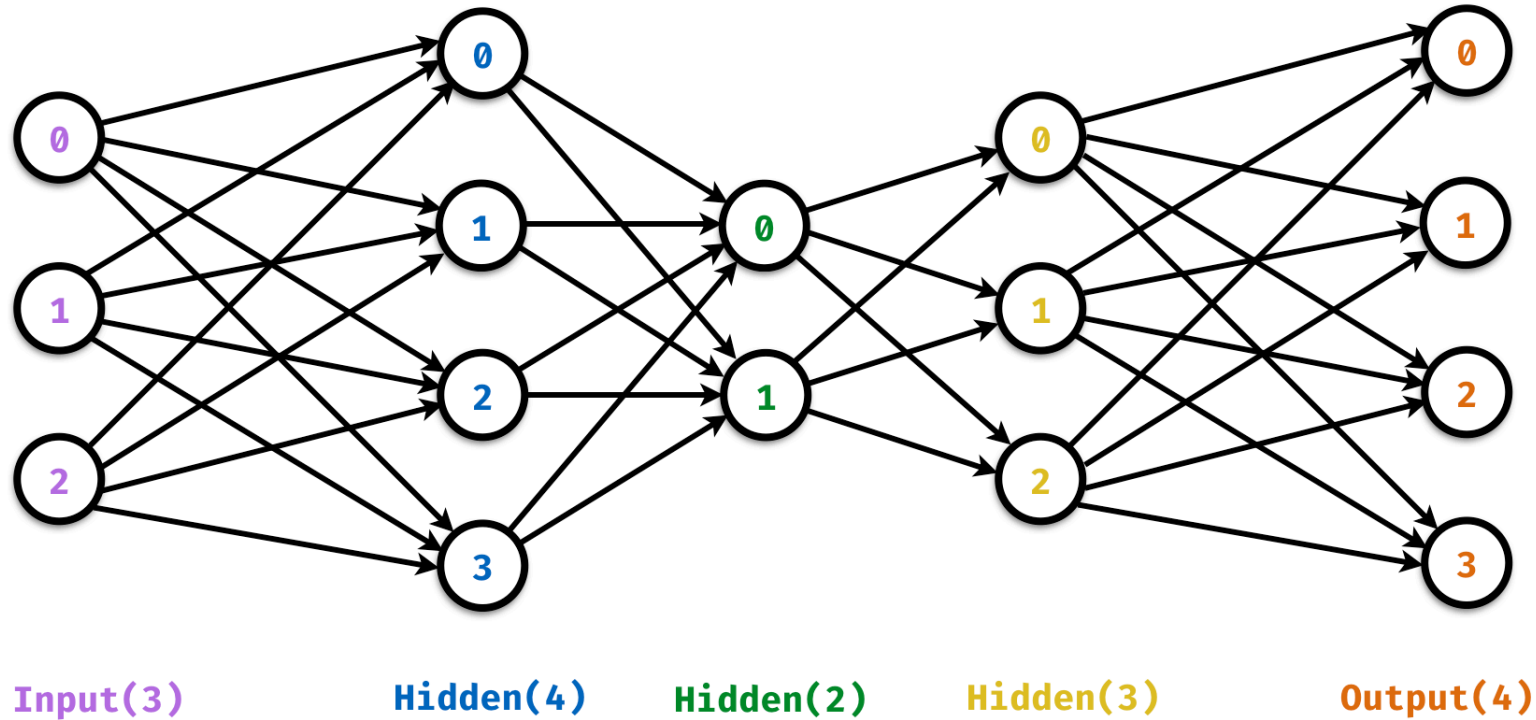
Neural Network has *Many* Layers



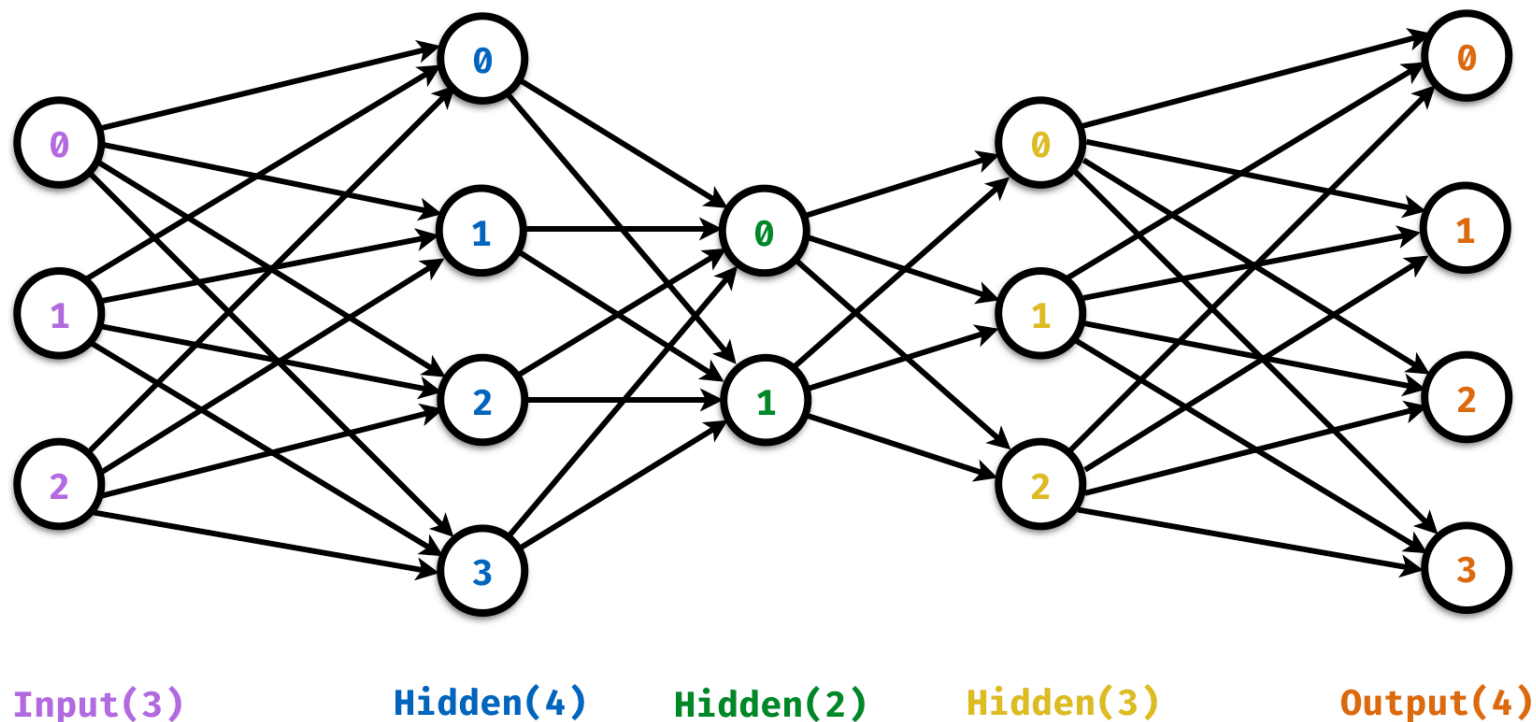
Neural Network has *Many* Layers



Neural Network has *Many* Layers



Neural Network has *Many* Layers



How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
enum Network {  
    Last(Layer),  
    Next(Layer, Box<Network>),  
}
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[@i, @o]) → Network[i, o],
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[@i, @o]) → Network[i, o],  
    Next(Layer[@i, @n], Box<Network[n, @o]>) → Network[i, o],  
}
```

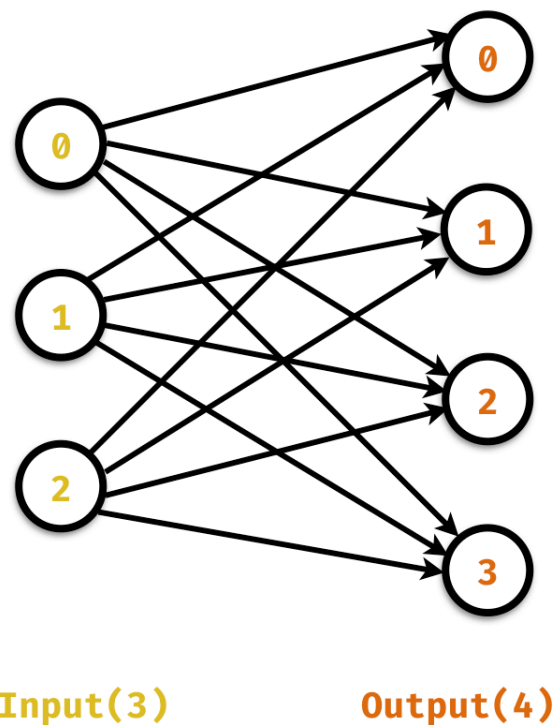
How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[@i, @o]) → Network[i, o],  
    Next(Layer[@i, @n], Box<Network[n, @o]>) → Network[i, o],  
}
```

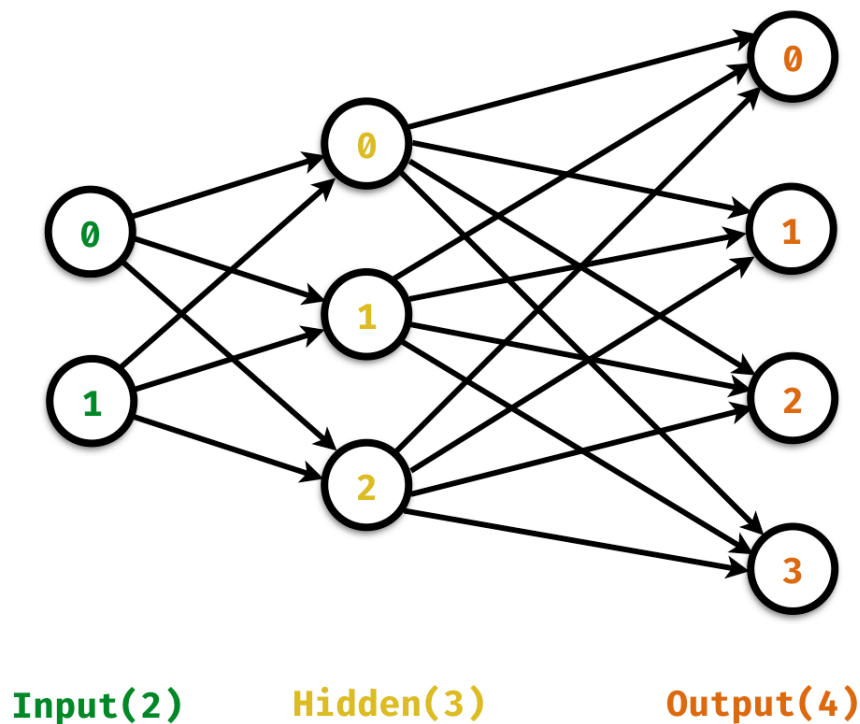
How to ensure layers *compose* correctly?

Refinements Ensure Correct *Composition*



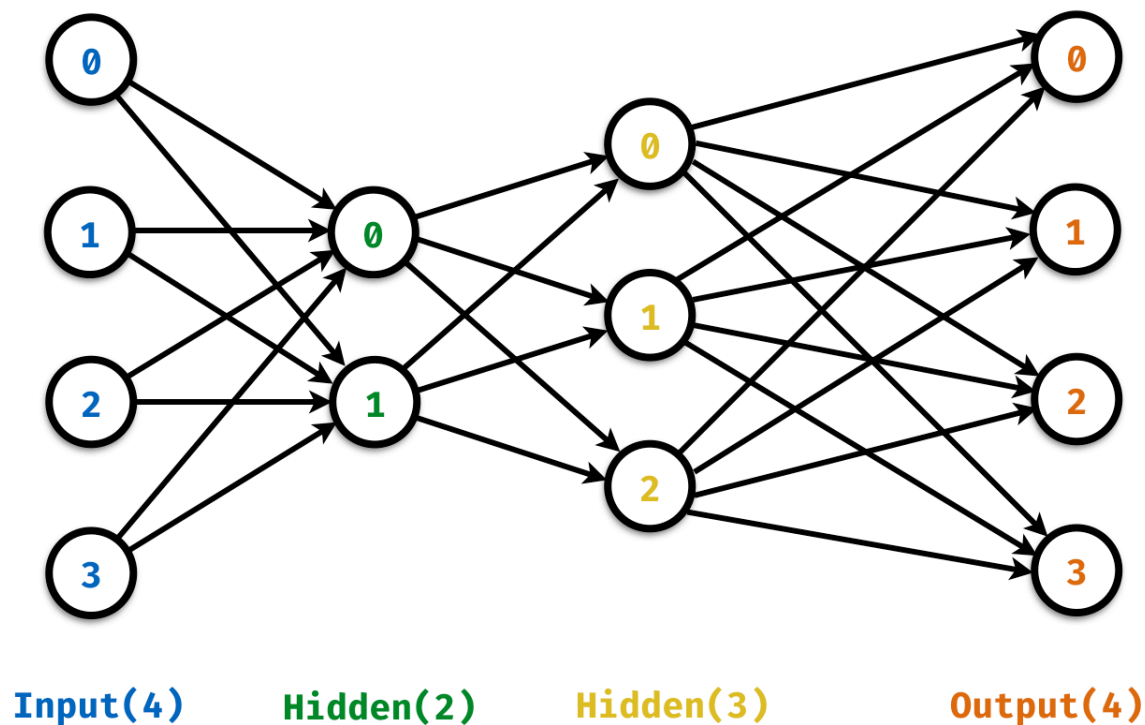
`Last(Layer[3,4])` $\longrightarrow$ `Network[3, 4]`

Refinements Ensure *Correct Composition*



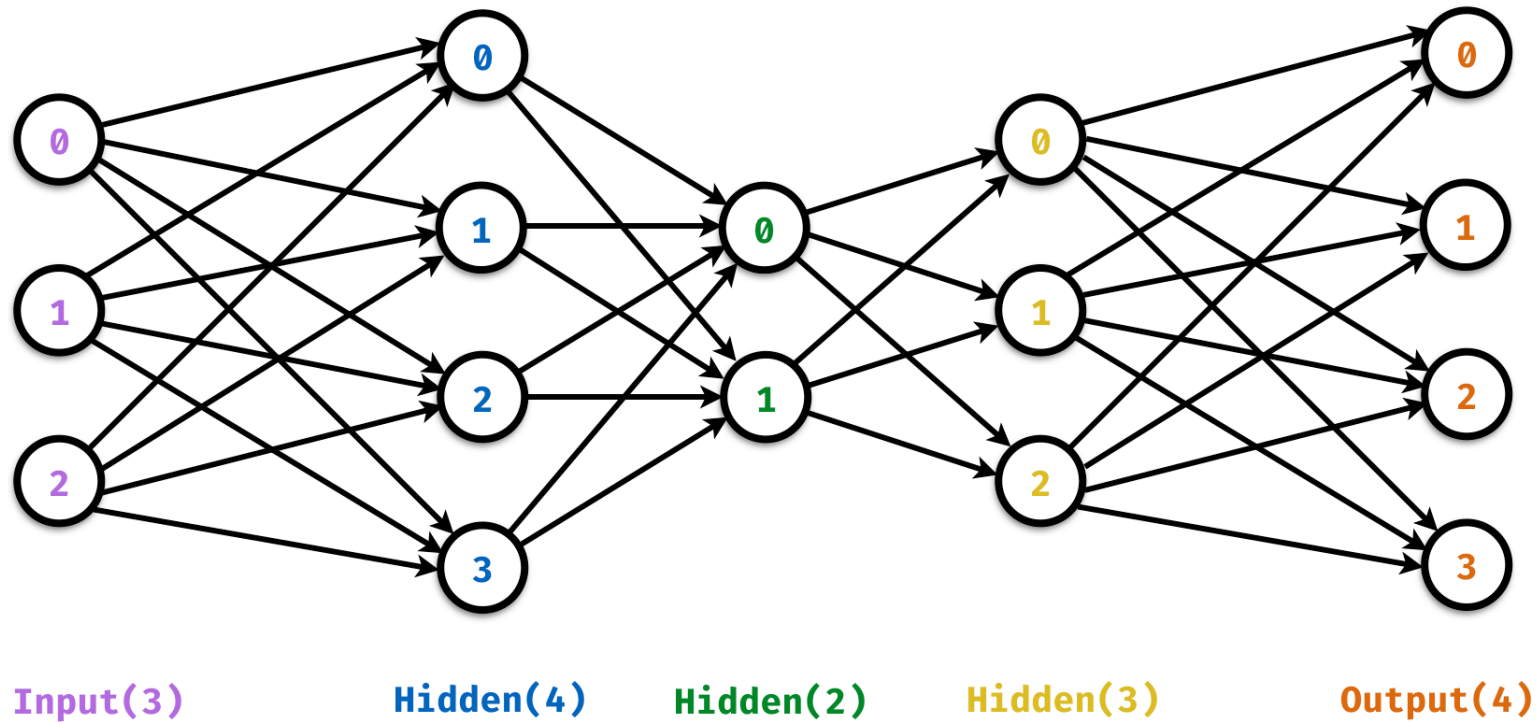
`Next(Layer[2,3], Network[3,4])  $\longrightarrow$  Network[2, 4]`

Refinements Ensure Correct *Composition*



`Next(Layer[4,2], Network[2,4])  $\longrightarrow$  Network[4, 4]`

Refinements Ensure Correct *Composition*



`Next(Layer[3,4], Network[4,4])  $\longrightarrow$  Network[4, 4]`

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)  
    → Network[input, output]
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    }
}
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
        Network::Next(layer, Box::new(rest))
    }
}
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
        Network::Next(layer, Box::new(rest))
    }
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
  }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {  
      layer.forward(input);  
    }  
  }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
    match self {  
        NeuralNetwork::Last(layer) ⇒ {  
            layer.forward(input);  
            layer.outputs.clone()  
        }  
        NeuralNetwork::Next(layer, next) ⇒ {  
            layer.forward(input);  
            next.forward(&layer.outputs)  
        }  
    }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {  
      layer.forward(input);  
      next.forward(&layer.outputs)  
    }  
  }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {  
      layer.forward(input);  
      next.forward(&layer.outputs)  
    }  
  }  
}
```

Exercise: *Fix* the specification for forward?

1. Problem

Program Logic vs. 21st century features

2: Solution

Refinements: *Weave* assertions into types

1. Problem

Program Logic *vs.* 21st century features

2: Solution

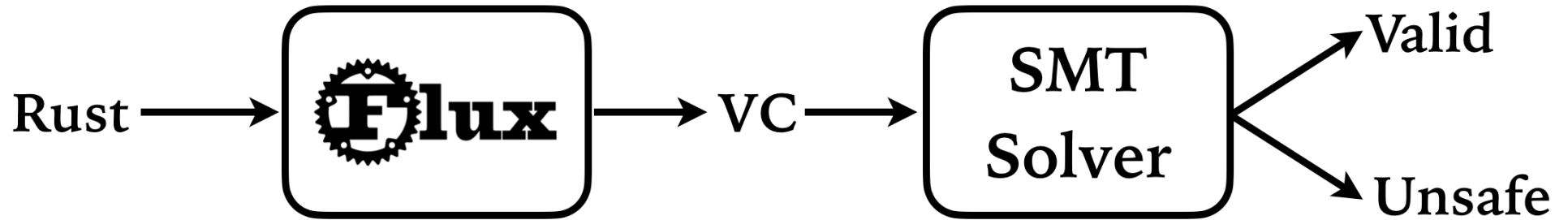
Refinements: *Weave* assertions into types

3: Speculation

Speculation

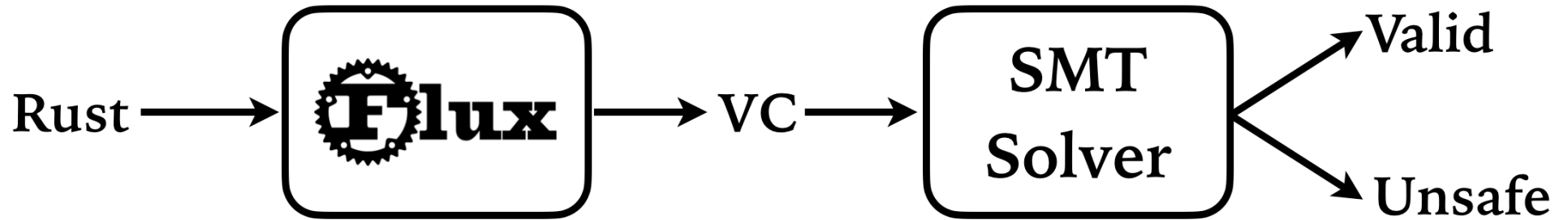
Refinements are *decidable* $\implies$ Typing is *automatic*

Speculation



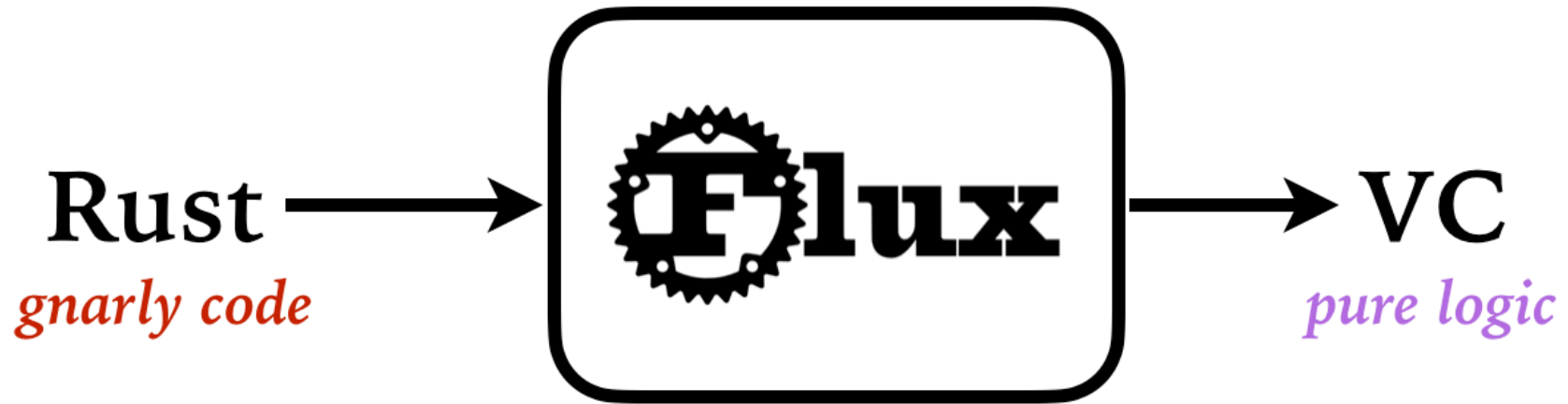
Refinements are *decidable* $\implies$ Typing is *automatic*

Speculation



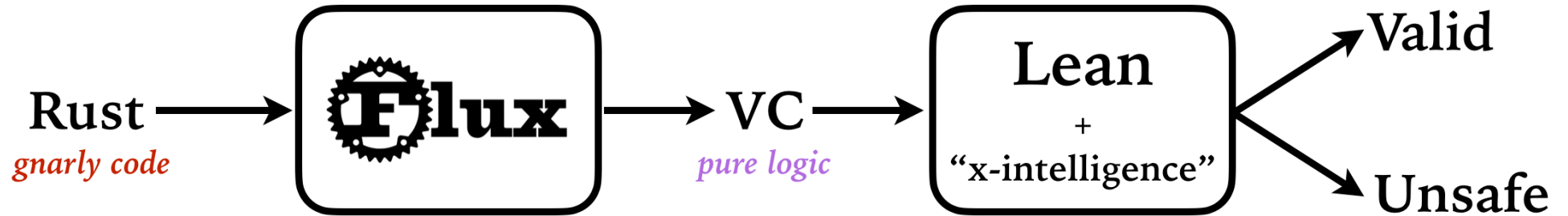
Refinements are *decidable* $\implies$ Expressiveness is *restricted*

Speculation



Petros' Idea: Flux compiles *impure* Rust to *pure* propositions!

Speculation



Recover expressiveness via *interaction* [Demo!]

1. Problem

Program Logic *vs.* 21st century features

2: Solution

Refinements: *Weave* assertions into types

3: Speculation

Lift code to propositions proved in Lean

Verification for the 21st Century

1. Problem

Program Logic *vs.* 21st century features

2: Solution

Refinements: Weave **assertions** into **types**

3: Speculation

Lift code to propositions proved in Lean



<https://flux-rs.github.io>